The worker's protocol is shown in the following snippet. You may be surprised to see two traits. Until now, you have only seen `Command`, but now you have `Response`. The type of actor is defined by the `Command` that defines the messages the actor exposes:

```
object Worker {

  sealed trait Command                                              ❶
  final case class Parse(replyTo: ActorRef[Worker.Response],        ❶
  ➥text: String)                                                   ❶
      extends Command                                               ❶

  sealed trait Response                                             ❷
  final case class Done(text: String) extends Response              ❷
  ...
}
```

❶ To be sent to the actor

❷ To be sent by the actor

Messages of type `Response` cannot be sent to the actor. They do not belong to its type. They must be sent by the actor that defines them to another actor, which means this other actor is responsible for translating and adapting the message. To finish, let's look at the worker.

3.1.8 Coding the worker

The logic of the worker is as follows:

1. The worker receives the `Parse` message.
2. It parses the text by removing any hyphens. (This example uses a simple function so we don't get distracted by the business logic.)